

Downcasting with Polymorphic Behaviour :

```
package com.ravi.downcasting;

class Animal
{
    public void roam()
    {
        IO.println("Generic Animal is roaming");
    }
}
class Dog extends Animal
{
    @Override
    public void roam()
    {
        IO.println("Dog Animal is roaming");
    }

    public void bark()
    {
        IO.println("Dog is barking");
    }
}
class Lion extends Animal
{
    @Override
    public void roam()
    {
        IO.println("Lion Animal is roaming");
    }

    public void roar()
    {
        IO.println("Lion is roaring");
    }
}
public class PolymorphicBehaviourDemo1
{
    public static void main(String[] args)
    {
        Animal a = new Dog(); //Up casting
        animalCamp(a);

        IO.println(".....");
        a = new Lion();
        animalCamp(a); //passing Lion object
    }

    public static void animalCamp(Animal animal) //loose coupling
    {
        if(animal instanceof Dog)
        {
            //Dog object is coming
            Dog dog = (Dog) animal;
            dog.roam();
            dog.bark();
        }
        else
        {
            //Lion object is coming
            Lion lion = (Lion) animal;
            lion.roam();
            lion.roar();
        }
    }
}
```

Downcasting with polymorphic behavior [Realtime example]

```
package com.ravi.downcasting;

class Payment
{
    public void makePayment(double amount)
    {
        IO.println("Making a generic payment");
    }
}
class UPI extends Payment
{
    @Override
    public void makePayment(double amount)
    {
        IO.println("Making a payment of "+amount+" using UPI");
    }

    public void offer()
    {
        IO.println("Payment done through UPI. Cash back amount is 500");
    }
}
class DebitCard extends Payment
{
    @Override
    public void makePayment(double amount)
    {
        IO.println("Making a payment of "+amount+" using Debit Card");
    }
}
class CreditCard extends Payment
{
    @Override
    public void makePayment(double amount)
    {
        IO.println("Making a payment of "+amount+" using Credit Card");
    }

    public void offer()
    {
        IO.println("2 Days holiday package in GOA");
    }
}
public class PolymorphicBehaviourDemo2
{
    public static void main(String[] args)
    {
        double amount = Double.parseDouble(IO.readLine("Enter the payment amount :"));

        Payment p = null;
        p = new UPI();
        paymentGateway(p, amount);

        IO.println(".....");

        p = new CreditCard();
        paymentGateway(p, amount);

        IO.println(".....");
        p = new DebitCard();
        paymentGateway(p, amount);
    }

    public static void paymentGateway(Payment payment, double amount)
    {
        if(payment instanceof UPI upi) //JDK 16
        {
            upi.makePayment(amount);
            upi.offer();
        }
        else if(payment instanceof CreditCard card) //JDK 16
        {
            card.makePayment(amount);
            card.offer();
        }
        else
        {
            DebitCard card = (DebitCard) payment; //Old approach
            card.makePayment(amount);
        }
    }
}
```

**** What is Method Hiding ?
OR
Can we override static method?

* In order to work with **method hiding** we have following cases :

Case 1 :

Any public static method of super class is by default available to sub class so from sub class we can call super class static method as shown in the program below :

```
class A
{
    public static void m1()
    {
        IO.println("static method m1() of A class ");
    }
}
class B extends A
{
}
public class MethodHiding
{
    public static void main(String[] args)
    {
        B b1 = new B();
        b1.m1();
    }
}
```

Case 2 :

* Method Hiding is only possible through static method as shown in the program

```
class A
{
    public static void m1()
    {
        IO.println("static method m1() of A class ");
    }
}
class B extends A
{
    public static int m1() //Incompatible return type with Method Hiding
    {
        IO.println("static method m1() of B class ");
        return 0;
    }
}
public class MethodHiding
{
    public static void main(String[] args)
    {
    }
}
```

Case 3 :

We can't override a static method with non static method because static method belongs to class and non static method belongs to object, If we try to override static method with non static method then it will generate an error i.e overridden method is static as shown below.

```
class A
{
    public static void m1()
    {
        IO.println("static method m1() of A class ");
    }
}
class B extends A
{
    public void m1() //error
    {
    }
}
public class MethodHiding
{
    public static void main(String[] args)
    {
    }
}
```

Case 4 :

We can't override any non static method with static method, If we try then it will generate an error, Overriding method is static.

```
class A
{
    public void m1()
    {
        IO.println("Non static method m1() of A class ");
    }
}
class B extends A
{
    public static void m1() //error
    {
    }
}
public class MethodHiding
{
    public static void main(String[] args)
    {
    }
}
```

Case 5 :

We can't override static method because it belongs to class but not object, If we write static method in the sub class with same signature and compatible return type then it is Method Hiding but not Method Overriding here compiler will search the method of super class and JVM will also execute the method of super class because method is not overridden. [Single copy and belongs to class area, sharable by all the objects]

Note :- 1) We can't apply @Override annotation on static methods.

2) Static methods can't be overridden so behavior is same for all the Objects hence it is STATIC Polymorphism.

```
class A
{
    public static void m1()
    {
        IO.println("Static method m1() of A class ");
    }
}
class B extends A
{
    public static void m1()
    {
        IO.println("Static method m1() of B class ");
    }
}
public class MethodHiding
{
    public static void main(String[] args)
    {
        A a1 = new B();
        a1.m1(); //Static method m1() of A class
    }
}
```

package com.ravi.method_hiding;

```
class Bird
{
    public static int eyes = 2;

    public static void fly()
    {
        IO.println("Bird is flying");
    }
}
class Parrot extends Bird
{
    public static int eyes = 2;

    public static void fly()
    {
        IO.println("Parrot is flying");
    }
}
class Peacock extends Bird
{
    public static int eyes = 2;

    public static void fly()
    {
        IO.println("Peacock is flying");
    }
}
public class MethodHiding
{
    public static void main(String[] args)
    {
        Bird b = new Parrot();      Note : In this program, if we create the object for any class but
        IO.println(b.eyes);          always Bird class static field and static method
        b.fly();                     will be executed. [Doesn't depend upon the object]
    }
}
```

MANTRA :

Non static Field, static Field and Static methods are always executed from the Current Reference